Pregunta De Implementación en Código (12 ptos.)

Una municipalidad necesita habilitar una mini-biblioteca digital para un barrio. Se requiere desarrollar **BiblioBarrio**, un servicio REST que gestione libros, usuarios y préstamos con autenticación segura. Su implementación **debe** usar arquitectura en 3 capas (Controller, Service, Repository), **DTOs** para entrada/salida, **manejo correcto de excepciones** y **seguridad JWT**.

Entidades

El sistema necesit a manejar los siguientes objetos de dominio:

User

```java
id: Long (autogenerado)
username: String (único, obligatorio)
email: String (único, obligatorio)
password: String (obligatorio, encriptado)
role: String (default: ROLE_READER)
```

Book

```java
id: Long (autogenerado)
title: String (obligatorio)
author: String (obligatorio)
isbn: String (único, opcional)
totalCopies: Integer (≥ 0)
availableCopies: Integer (≥ 0)
```

Loan

```java
id: Long (autogenerado)
bookId: Long (referencia a Book)
userId: Long (referencia a User)
borrowDate: LocalDate (ISO-8601; por defecto hoy)
dueDate: LocalDate (ISO-8601; regla: borrowDate + 14 días)
status: String (ACTIVE, RETURNED, OVERDUE)
```

Reservation

```java
id: Long (autogenerado)
bookId: Long (referencia a Book)
userId: Long (referencia a User)
reservedAt: ZonedDateTime
expiresAt: ZonedDateTime (reservedAt + 48 horas)
status: String (PENDING, EXPIRED, CANCELLED)
```

Endpoints

1. **Registro de Usuario**: POST /auth/register

```json
// Request DTO
{
  "username": "maria.lopez",
  "email": "maria@ejemplo.com",
  "password": "Biblio2025"
}

// Response DTO
{
  "id": 1,
  "username": "maria.lopez",
  "email": "maria@ejemplo.com"
}
```

Status Code: Created

Validaciones:

- username/email únicos
- password ≥ 8 caracteres

2. **Autenticación:** POST /auth/login

```json
// Request DTO
{
```

```
  "username": "maria.lopez",
  "password": "Biblio2025"
}

// Response DTO
{
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "expiresIn": 3600
}
```

Status Code: 200 OK

3. **Crear Libro:** POST /books

```
JSON
// Request DTO
{
  "title": "Cien años de soledad",
  "author": "Gabriel García Márquez",
  "isbn": "978-0307474728",
  "totalCopies": 3
}

// Response DTO
{
  "id": 1,
  "title": "Cien años de soledad",
  "author": "Gabriel García Márquez",
  "isbn": "978-0307474728",
  "totalCopies": 3,
  "availableCopies": 3
}
```

Headers:

- **Authorization:** Bearer {{token}}
- **Content-Type:** application/json

Status Code: Created

Validaciones:

- isbn único (si se proporciona)
- totalCopies ≥ 0

4. **Listar Libros:** GET /books

```JSON
// Response DTO (status=available)
{
  "content": [
    {
      "id": 1,
      "title": "Cien años de soledad",
      "author": "Gabriel García Márquez",
      "availableCopies": 2,
      "totalCopies": 3
    },
    {
      "id": 2,
      "title": "1984",
      "author": "George Orwell",
      "availableCopies": 1,
      "totalCopies": 2
    }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 15
}
```

Headers:

- **Authorization:** Bearer {{token}}

Query Parameters:

- **status:** all | available | unavailable (default: all)
- **page:** 0 (default)
- **size:** 10 (default)

Status Code: OK

Lógica:

- **all:** mostrar todos los libros
- **available:** mostrar sólo libros con availableCopies > 0
- **unavailable:** mostrar sólo libros con availableCopies = 0

5. **Crear Préstamo:** POST /loans

```JSON
// Request DTO
{
  "bookId": 1,
  "borrowDate": "2025-10-05"
}

// Response DTO
{
  "id": 10,
  "bookId": 1,
  "userId": 1,
  "borrowerName": "maria.lopez",
  "borrowDate": "2025-10-05",
  "dueDate": "2025-10-19",
  "status": "ACTIVE"
}
```

Headers:

- **Authorization:** Bearer {{token}}
- **Content-Type:** application/json

Status Code: Created

Lógica:

- Decrementar availableCopies del libro
- userId se obtiene del token JWT
- Validar que el usuario no tenga préstamos vencidos

6. **Crear Reserva:** POST /reservations

```JSON
// Request DTO
{
  "bookId": 1
}

// Response DTO
{
  "id": 5,
  "bookId": 1,
  "bookTitle": "Cien años de soledad",
  "userId": 1,
  "userName": "maria.lopez",
  "reservedAt": "2025-10-05T14:30:00-05:00",
  "expiresAt": "2025-10-07T14:30:00-05:00",
  "status": "PENDING"
}
```

Headers:

- **Authorization:** Bearer {{token}}
- **Content-Type:** application/json

Status Code: Created

Lógica:

- Solo se puede reservar si availableCopies = 0
- La reserva expira en 48 horas
- Usuario no puede tener más de 3 reservas activas

7. **Consultar Mi Actividad:** GET /my-activity

```JSON
// Response DTO (type=all)
{
  "content": [
    {
      "type": "LOAN",
      "id": 10,
```

```
      "bookId": 1,
      "bookTitle": "Cien años de soledad",
      "borrowDate": "2025-10-05",
      "dueDate": "2025-10-19",
      "status": "ACTIVE"
    },
    {
      "type": "RESERVATION",
      "id": 5,
      "bookId": 2,
      "bookTitle": "1984",
      "reservedAt": "2025-10-06T10:00:00-05:00",
      "expiresAt": "2025-10-08T10:00:00-05:00",
      "status": "PENDING"
    }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 5
}
```


Headers:

- **Authorization:** Bearer {{token}}

Query Parameters:

- **type:** all | loans | reservations (default: all)
- **page:** 0 (default)
- **size:** 10 (default)

Status Code: OK

Lógica:

- **all:** mostrar tanto préstamos como reservas del usuario autenticado
- **loans:** mostrar solo préstamos (ACTIVE, RETURNED, OVERDUE)
- **reservations:** mostrar solo reservas (PENDING, EXPIRED, CANCELLED)
- Ordenar por fecha de creación descendente (más reciente primero)

Excepciones

La implementación debe contemplar el uso de por lo menos las siguientes excepciones:

- **UserNotFoundException → 404 Not Found**
    - Caso: operaciones con userId inexistente
- **UserAlreadyExistsException → 409 Conflict**
    - Caso: POST /auth/register con username o email ya registrado
- **BookNotFoundException → 404 Not Found**
    - Caso: GET/DELETE /books/{id} o préstamo/reserva con bookId inexistente
- **NoCopiesAvailableException → 409 Conflict**
    - Caso: POST /loans cuando availableCopies = 0
- **LoanNotFoundException → 404 Not Found**
    - Caso: PUT /loans/{id}/return con id inexistente
- **OverdueLoanException → 400 Bad Request**
    - Caso: intentar crear préstamo cuando el usuario tiene préstamos vencidos